

UXL FOUNDATION

UXL Skills Evaluator

Project-maintained operational knowledge for coding agents

Maintainer overview • September 2026

What exists today

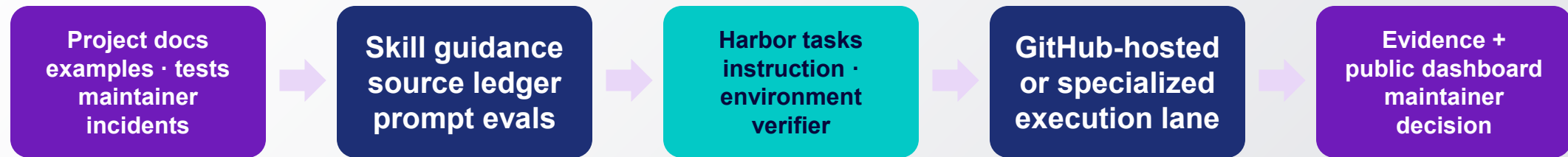
- **8 skills** — Six library portfolios plus shared SYCL build/debug and performance-validation guidance.
- **52 scoped evaluation tasks** — 41 implemented and 11 left visible as planned gaps; this is inventory, not a pass-rate claim.
- **A public control room** — Skills, evaluations, platforms, methodology, maturity, source freshness, and ownership status.
- **A portable execution model** — GitHub-hosted checks by default; opt-in specialized machines only when a task contract requires them.

The current skill catalog — at a glance

- **oneDNN** — Primitives, layouts/reorders, fusion, numerics, benchdnn.
- **oneMath** — Domains, dispatch modes, queues, backends, linking.
- **oneDAL** — Python/native paths, modes, tables, metric parity, conversion cost.
- **oneTBB** — Patterns, shared state, flow graphs, arenas, grainsize/affinity.
- **oneDPL** — Host/device policies, iterators, lifetime, synchronization, ordering.
- **oneCCL** — Collectives, ranks, launch symmetry, waits, plugins, hangs.
- **Cross-project** — SYCL build/runtime triage and correctness-first performance evidence.

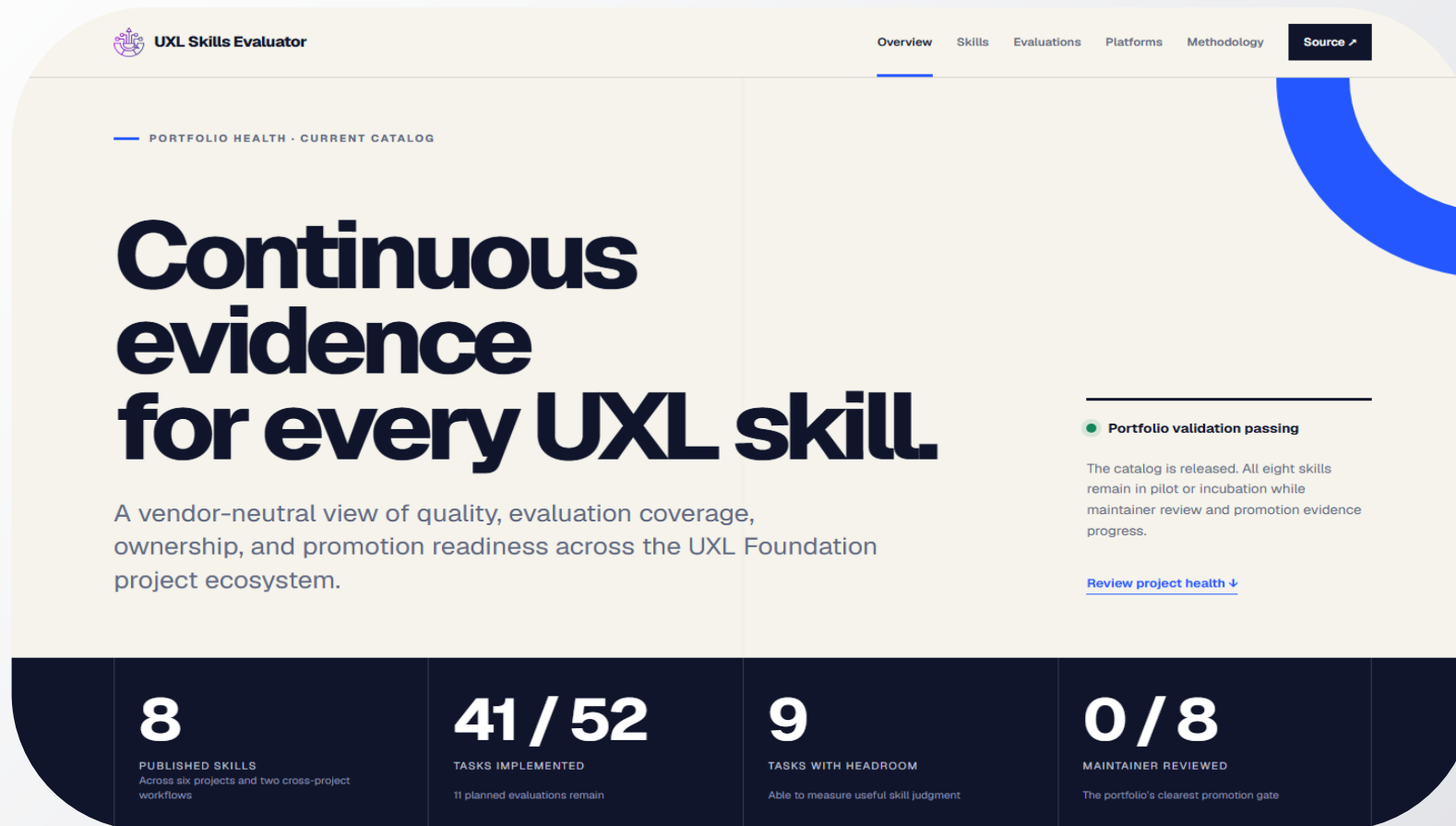
Architecture: project knowledge becomes measured evidence

One repository carries guidance, task contracts, policy, and the public portfolio view.

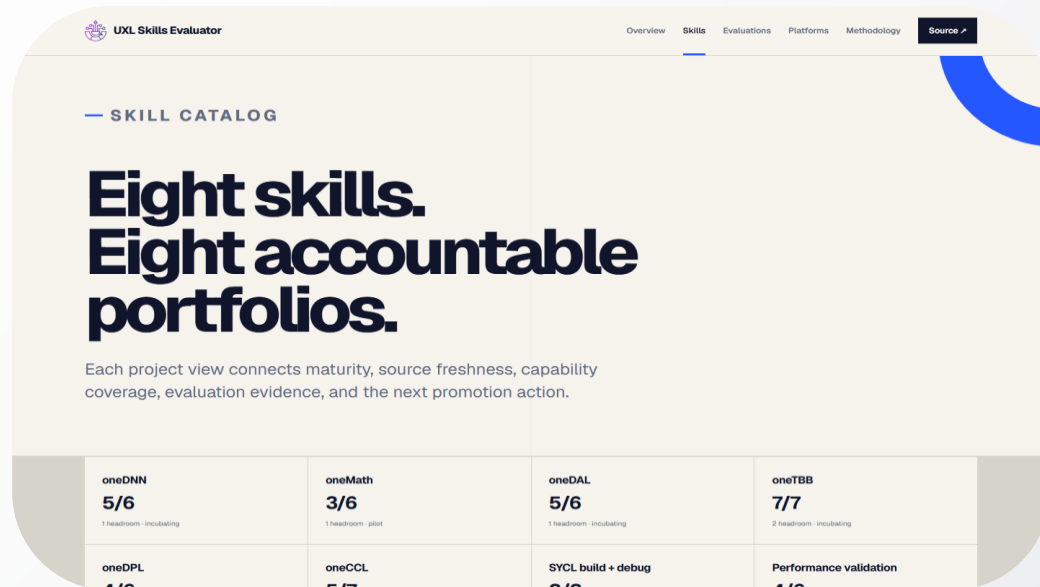


Specialized machines change the environment, not the evidence contract.

The public control room answers: are the skills healthy?

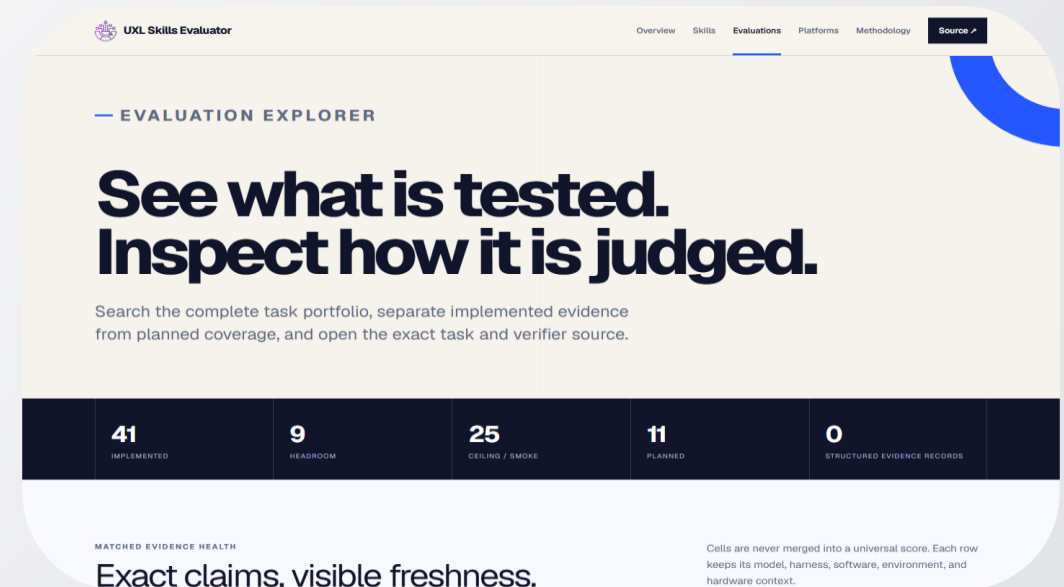


Maintainers can drill from portfolio to exact task contracts



oneDNN 5/6 1 headroom - incubating	oneMath 3/6 1 headroom - pilot	oneDAL 5/6 1 headroom - incubating	oneTBB 7/7 2 headroom - incubating
oneDPL A 12	oneCCL E 17	SYCL build + debug O 10	Performance validation A 12

Skills: scope, maturity, sources, limitations



41 IMPLEMENTED	9 HEADROOM	25 CEILING / SMOKE	11 PLANNED	0 STRUCTURED EVIDENCE RECORDS
-------------------	---------------	-----------------------	---------------	----------------------------------

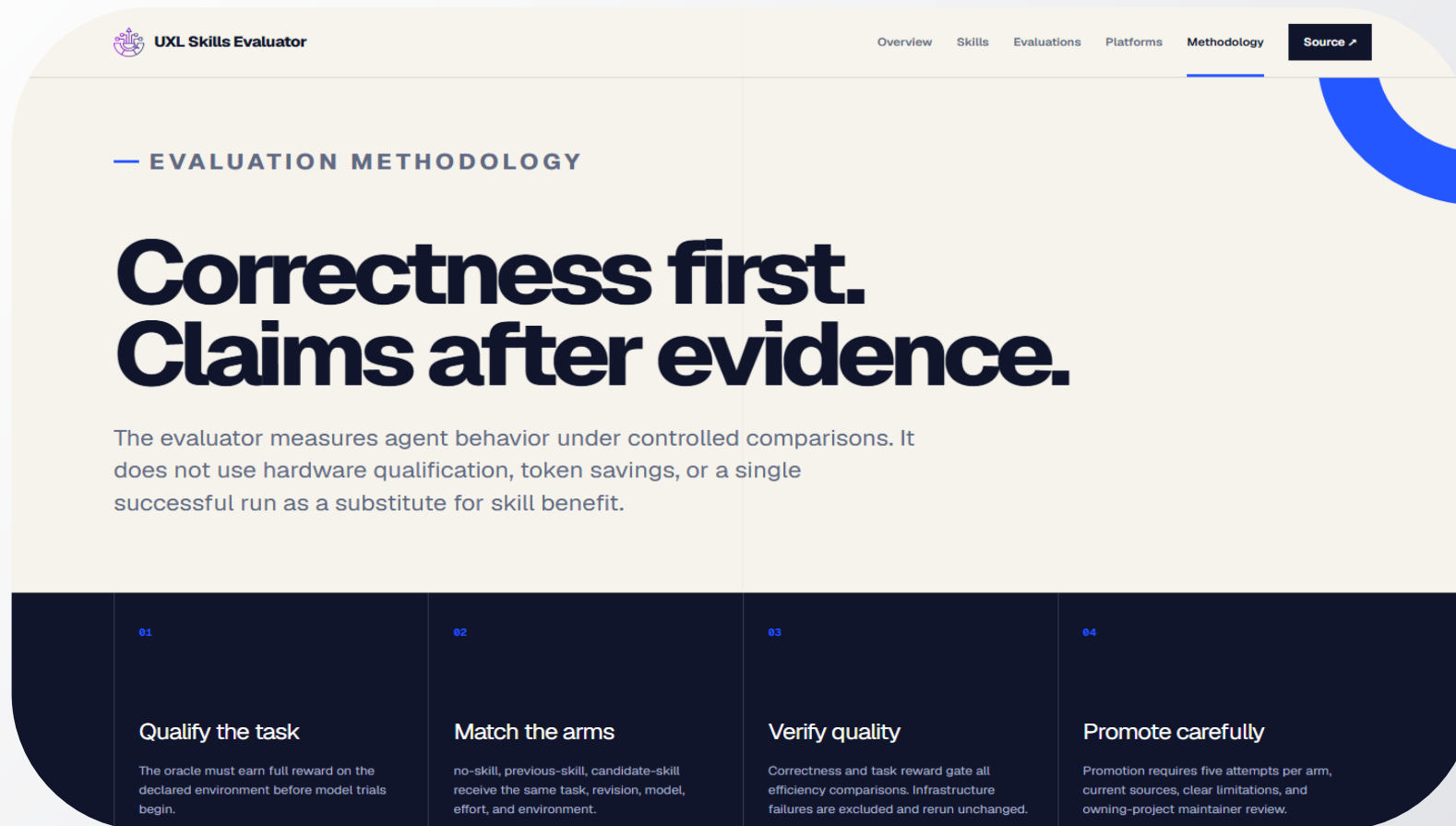
MATCHED EVIDENCE HEALTH

Exact claims, visible freshness.

Cells are never merged into a universal score. Each row keeps its model, harness, software, environment, and hardware context.

Evaluations: exact tasks, states, environments

The methodology is deliberately claim-resistant

The image is a screenshot of the 'UxL Skills Evaluator' website, specifically the 'Methodology' page. The page has a light beige background with a dark blue header and footer. The header includes the UXL logo and the text 'UxL Skills Evaluator' on the left, and a navigation menu with links for 'Overview', 'Skills', 'Evaluations', 'Platforms', 'Methodology' (which is highlighted with a blue underline), and 'Source' on the right. Below the header, the page title 'EVALUATION METHODOLOGY' is displayed in a small, dark blue font. The main content area features a large, bold, dark blue heading: 'Correctness first. Claims after evidence.' Below this heading, a paragraph of text explains the methodology: 'The evaluator measures agent behavior under controlled comparisons. It does not use hardware qualification, token savings, or a single successful run as a substitute for skill benefit.' At the bottom of the page, there is a dark blue horizontal bar containing four white boxes, each representing a step in the methodology. Each box has a small blue number in the top left corner and a title followed by a brief description. The steps are: 01 'Qualify the task' (The oracle must earn full reward on the declared environment before model trials begin.), 02 'Match the arms' (no-skill, previous-skill, candidate-skill receive the same task, revision, model, effort, and environment.), 03 'Verify quality' (Correctness and task reward gate all efficiency comparisons. Infrastructure failures are excluded and rerun unchanged.), and 04 'Promote carefully' (Promotion requires five attempts per arm, current sources, clear limitations, and owning-project maintainer review.).

UxL Skills Evaluator

Overview Skills Evaluations Platforms Methodology Source ↗

— EVALUATION METHODOLOGY

Correctness first. Claims after evidence.

The evaluator measures agent behavior under controlled comparisons. It does not use hardware qualification, token savings, or a single successful run as a substitute for skill benefit.

01	02	03	04
Qualify the task The oracle must earn full reward on the declared environment before model trials begin.	Match the arms no-skill, previous-skill, candidate-skill receive the same task, revision, model, effort, and environment.	Verify quality Correctness and task reward gate all efficiency comparisons. Infrastructure failures are excluded and rerun unchanged.	Promote carefully Promotion requires five attempts per arm, current sources, clear limitations, and owning-project maintainer review.

Every claim lives inside a matched evaluation cell

Inside one cell, change only the skill treatment.



Repeat across representative cells; report differences. Never average incompatible contexts into a universal score.

Maintainer pushback is expected — and the answer is scoped evidence

- Do you test every combination? — No. We publish representative, risk-based cells and the gaps.
- What changes invalidate evidence? — Re-run affected cells; older results remain history, not current proof.
- How do you limit overfitting? — Maintainer incidents, hidden checks, negative controls, multiple tasks, and trajectory review.
- What if every arm passes? — Keep it as smoke coverage; claim no lift and seek a harder task.
- Does hardware qualification prove skill value? — No. Qualification and matched skill trials are separate gates.
- Can scores be pooled? — No universal score; report conclusions within each model, harness, software, and environment cell.

Hardware is a pluggable evidence lane — never the product

UXL Skills Evaluator

Overview Skills Evaluations **Platforms** Methodology Source ↗

— PLATFORM EVIDENCE

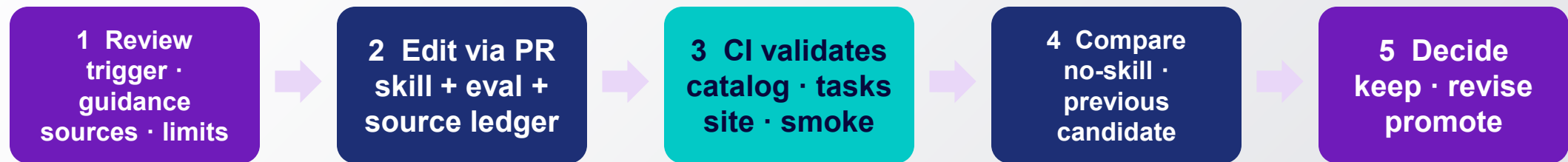
Evidence contract first. Hardware second.

Hosted systems remain the default. Specialized machines join only when a task requires a particular device, backend, topology, driver, or instruction set.

01	02	03
Vendor neutral No platform receives preferred status. The same task, verifier, revision, and provenance rules apply everywhere.	Capability matched A task moves to specialized hardware only when hosted infrastructure cannot reproduce the behavior faithfully.	Evidence portable Approved systems return the same Harbor result structure, so evaluation remains comparable and reviewable.

The ongoing maintainer workflow fits normal GitHub practice

A maintainer's recurring action is a small review — the shared evaluator does the rest.



Quality is the gate; lower token use never compensates for fewer verified successes.

Ownership is intentionally split

- **Project maintainers own** — Triggers, technical guidance, official sources, limitations, and whether task scenarios are authentic.
- **UXL shared infrastructure owns** — Catalog validation, Harbor harnesses, comparison policy, dashboards, hosted CI, and runner contracts.
- **Hardware owners opt in** — They qualify a declared environment, run only reviewed immutable evaluator revisions, and return evidence artifacts.
- **No silent promotion** — Project approval, source verification, realistic eval prompts, matched trials when feasible, and explicit limitations remain gates.

The decision we need from each project

- **Name one owner** — A maintainer or small reviewer group for scope and source freshness.
- **Approve or correct the first-pass guidance** — Especially triggers, gotchas, unsupported paths, and terminology.
- **Validate the task inventory** — Keep, rewrite, or delete scenarios; nominate one real support issue or regression.
- **Adopt a light cadence** — Review skill-affecting PRs and recheck sources at releases or quarterly.
- **Start small** — A 30-minute review is enough to turn this from UXL-authored draft into project-backed guidance.